give me the damn blueprint

Here is the complete technical blueprint. No fluff, no filler.

⬛ LIVE M&A DUE DILIGENCE WAR ROOM AGENT

Complete Technical Blueprint — Google x Columbia Hackathon 2026

Version: 1.0 | **Build Window:** 6 hours | **Track:** Live Agent

PART 1: STRATEGIC OVERVIEW

What This Is

A real-time, voice-driven M&A due diligence system where a user speaks a company name into a microphone and receives a live, spoken briefing from a senior investment banker AI — backed by three parallel research sub-agents running simultaneously, synthesized by a Live API orchestrator, and written to Firestore as a structured deal memo.

Why This Wins

- Uses **Gemini Live API** (real-time voice, barge-in capable) ✅
- Uses **ADK ParallelAgent** (3 simultaneous sub-agents) ✅
- Uses **Google Search Grounding** (real-time financial data) ✅
- Uses **Firestore** (live document writes) ✅
- Uses **Cloud Run** (deployed backend) ✅
- Not a chatbot, not a RAG app, not text-in/text-out ✅
- Demo is self-evidently jaw-dropping: someone speaks "analyze Apple" and 45 seconds later gets a verbal banker briefing with red flags called out ✅

Prohibited Check

- ❌ AI Mental Health/Medical Advisor → NOT this
- ❌ Basic RAG ("Chat with my PDF") → NOT this (no PDF, live web grounding)
- ❌ Standard Education Chatbot → NOT this

- ❌ Basic Image Analyzer → NOT this
- ❌ Text-In, Text-Out only → NOT this (voice in, voice out, Firestore write)
- ✅ All clear

PART 2: SYSTEM ARCHITECTURE

High-Level Architecture

```
┌──────────────────────────────────────────────────────────────┐
│                        USER INTERFACE                        │
│              Next.js Frontend (Firebase Hosting)              │
│   [Mic Button] → WebSocket → [Speaker Output + Text Feed]     │
└──────────────────────────┬───────────────────────────────────┘
                           │ WebSocket (ws://)
┌──────────────────────────▼───────────────────────────────────┐
│                    FASTAPI BACKEND                            │
│                  (Cloud Run Container)                        │
│                                                               │
│  /ws  ⟵ WebSocket Handler                                     │
│          ↓                                                    │
│  Audio Buffer → Gemini Live API Session                       │
│                (Bidirectional streaming)                      │
│          ↓                                                    │
│  Intent Detected: "Analyze [COMPANY]"                         │
│          ↓                                                    │
│  ADK Orchestrator Agent (LlmAgent)                            │
│      ↓                                                        │
│  ParallelAgent → [Agent A] [Agent B] [Agent C]                │
│      ↓                                                        │
│  Synthesis Agent → Spoken Briefing + Firestore Write          │
└──────────────────────────┬───────────────────────────────────┘
                           │
          ┌────────────────┼────────────────┐
          ▼                ▼                ▼
   ┌──────────┐      ┌──────────┐      ┌──────────────┐
   │Agent A   │      │Agent B   │      │Agent C       │
   │Financial │      │Competi-  │      │Sentiment/    │
   │Analyst   │      │tive Intel│      │Risk Analyst  │
   │          │      │          │      │              │
   │Google    │      │Google    │      │Google Search │
   │Search    │      │Search    │      │Grounding +   │
   │Grounding │      │Grounding │      │Long Context  │
   └──────────┘      └──────────┘      └──────────────┘
          └────────────────┼────────────────┘
                           │
                  ┌────────▼────────┐
                  │   Firestore DB  │
                  │ /deals/{session}│
                  │ - financials    │
                  │ - competitive   │
                  │ - sentiment     │
                  │ - red_flags     │
```

```
              │  - deal_memo       │
              └────────────────────┘
```

Data Flow Sequence

```
1. User opens app → WebSocket connection established
2. User clicks mic → Audio stream begins
3. Gemini Live API receives audio chunks in real-time
4. Live API transcribes + understands intent
5. Live API detects company name trigger
6. Backend fires ADK ParallelAgent with company name
7. Three agents run simultaneously (~15-20 seconds):
   - Agent A: Revenue, margins, cash flow, debt, recent filings
   - Agent B: Market position, competitors, moat analysis
   - Agent C: Sentiment from news, earnings calls, red flag detection
8. Results merge into shared session state
9. Synthesis Agent crafts banker-style verbal briefing
10. Live API speaks briefing back to user in real-time
11. Firestore receives structured deal memo simultaneously
12. Frontend shows live text feed of briefing + memo link
13. User can interrupt at any time → Live API handles barge-in
14. Follow-up questions answered from already-loaded context
```

PART 3: COMPLETE FILE STRUCTURE

```
ma-warroom/
├── backend/
│   ├── main.py                  # FastAPI app + WebSocket handler
│   ├── live_session.py          # Gemini Live API session manager
│   ├── agents/
│   │   ├── __init__.py
│   │   ├── orchestrator.py       # Root ADK LlmAgent + ParallelAgent
│   │   ├── financial_agent.py    # Agent A: Financial analysis
│   │   ├── competitive_agent.py   # Agent B: Competitive landscape
│   │   ├── sentiment_agent.py     # Agent C: Sentiment + risk
│   │   └── synthesis_agent.py    # Synthesis + briefing generator
│   ├── services/
│   │   ├── __init__.py
│   │   ├── firestore_service.py   # Firestore write/read operations
│   │   └── audio_utils.py        # Audio format conversion helpers
│   ├── prompts/
│   │   ├── orchestrator_prompt.py
│   │   ├── financial_prompt.py
│   │   ├── competitive_prompt.py
│   │   ├── sentiment_prompt.py
│   │   └── synthesis_prompt.py
│   ├── requirements.txt
│   ├── Dockerfile
│   └── .env
├── frontend/
│   ├── app/
│   │   ├── page.tsx              # Main war room UI
```

```
│   │   ├── layout.tsx
│   │   └── globals.css
│   ├── components/
│   │   ├── MicButton.tsx         # Voice input control
│   │   ├── BriefingFeed.tsx       # Live text feed of spoken briefing
│   │   ├── DealMemo.tsx          # Structured memo display
│   │   ├── AgentStatus.tsx       # Live agent status indicators
│   │   └── RedFlagAlert.tsx      # Red flag popup component
│   ├── hooks/
│   │   └── useWarRoom.ts         # WebSocket + audio logic
│   ├── lib/
│   │   └── firestore.ts          # Firestore client reads
│   ├── package.json
│   └── .env.local
├── firestore.rules               # Firestore security rules
├── firebase.json                 # Firebase config
└── README.md
```

PART 4: BACKEND — COMPLETE CODE

`backend/requirements.txt`

```
fastapi==0.115.0
uvicorn[standard]==0.32.0
websockets==13.1
google-adk==1.0.0
google-genai==1.5.0
google-cloud-firestore==2.19.0
python-dotenv==1.0.1
asyncio==3.4.3
aiohttp==3.10.10
pydantic==2.9.2
```

`backend/.env`

```
# Google AI Studio API Key (for local dev)
GOOGLE_API_KEY=your_google_ai_studio_key_here

# OR for Vertex AI (for Cloud Run deployment)
GOOGLE_GENAI_USE_VERTEXAI=FALSE
GOOGLE_CLOUD_PROJECT=your_gcp_project_id
GOOGLE_CLOUD_LOCATION=us-central1

# Firestore
FIRESTORE_PROJECT_ID=your_gcp_project_id

# App config
ENVIRONMENT=development
PORT=8000
```

`backend/prompts/orchestrator_prompt.py`

```
ORCHESTRATOR_SYSTEM_PROMPT = """
You are the Chief M&A Orchestrator for an elite investment banking war room.

Your role:
1. Listen for a company name from the user's voice input
2. When you detect a company name, extract it and trigger parallel research
3. Synthesize the research findings from your three specialist agents
4. Deliver a crisp, spoken senior banker briefing — authoritative, specific, and red-fl

Tone: Senior Goldman Sachs MD. Precise. No fluff. Lead with the most important finding.
Every number you cite must come from grounded search data.

When the user interrupts or asks a follow-up question mid-briefing:
- Stop immediately and address the question
- Reference only data already loaded in session state
- Return to briefing if user says "continue"

Format your spoken output as:
"[Company] overview: [2-sentence macro picture]. Financials: [key metrics + 1 flag if an
Competitive position: [moat assessment + key threat]. Risk signals: [top 1-2 risks].
Overall: [buy/watch/avoid recommendation with reasoning]."

Never say "I found" or "Based on my search." Speak as if you know this company cold.
"""
```

`backend/prompts/financial_prompt.py`

```
FINANCIAL_ANALYST_PROMPT = """
You are a buy-side financial analyst. Your job is to research a single company's
financial health using Google Search grounding. Execute immediately — no preamble.

For the given company, find and return structured data on:
1. Revenue (last 2 years, YoY growth %)
2. EBITDA margin (current vs industry average if findable)
3. Net income / operating cash flow trend
4. Debt-to-equity ratio or net debt figure
5. Most recent earnings surprise (beat/miss + %)
6. Any guidance cuts, restatements, or auditor flags

CRITICAL: Flag any discrepancy between management narrative and actual reported numbers.
Flag if Q3/Q4 cash flow diverges significantly from revenue trend.
Flag any recent CFO/auditor changes.

Return as structured JSON with keys:
{
  "revenue": {"current": "", "prior_year": "", "growth_pct": ""},
  "ebitda_margin": {"current": "", "industry_avg": ""},
  "cash_flow": {"operating": "", "trend": "improving|declining|stable"},
  "debt": {"de_ratio": "", "net_debt": ""},
```

```
  "earnings": {"last_surprise": "", "beat_miss": ""},
  "red_flags": ["flag1", "flag2"],
  "sources": ["url1", "url2"]
}
"""
```

backend/prompts/competitive_prompt.py

```
COMPETITIVE_ANALYST_PROMPT = """
You are a competitive intelligence analyst. Research the competitive landscape
for the given company using Google Search grounding. No preamble — execute immediately

Find and return:
1. Top 3 direct competitors with market share estimates
2. Company's primary competitive moat (cost, network, brand, IP, switching cost)
3. Most recent competitive threat or market share shift
4. Any recent product launch, acquisition, or strategic pivot by competitors
5. Industry tailwind/headwind (1 sentence)

CRITICAL: Note if any competitor just raised significant funding or announced a product
that directly threatens this company's core revenue line.

Return as structured JSON:
{
  "competitors": [
    {"name": "", "market_share_est": "", "key_differentiator": ""}
  ],
  "moat": {"type": "", "strength": "strong|moderate|weak", "rationale": ""},
  "recent_threat": {"company": "", "action": "", "impact": ""},
  "industry_trend": "",
  "red_flags": ["flag1"],
  "sources": ["url1"]
}
"""
```

backend/prompts/sentiment_prompt.py

```
SENTIMENT_ANALYST_PROMPT = """
You are a risk and sentiment analyst. Using Google Search grounding, analyze
recent news, executive statements, and public signals for the given company.
No preamble — execute immediately.

Find and return:
1. Overall news sentiment (last 30 days): positive/neutral/negative + brief reason
2. Any executive departures (C-suite level) in last 6 months
3. Regulatory, legal, or SEC investigation mentions
4. Recent layoffs, restructuring, or office closures
5. Analyst rating changes (upgrades/downgrades) in last 30 days
6. Social/media sentiment flags (viral negative press, controversy, boycotts)

CRITICAL FLAGS:
```

```
- CEO/CFO departure within 90 days = HIGH risk flag
- SEC investigation mention = HIGH risk flag
- Guidance cut + insider selling = HIGH risk flag
- Any mention of "going concern" = CRITICAL flag

Return as structured JSON:
{
  "news_sentiment": {"score": "positive|neutral|negative", "reason": ""},
  "executive_changes": [{"role": "", "type": "departure|appointment", "date": ""}],
  "regulatory": {"issues": "", "severity": "none|low|medium|high"},
  "restructuring": {"occurred": true/false, "details": ""},
  "analyst_ratings": {"recent_changes": "", "consensus": ""},
  "red_flags": ["flag1", "flag2"],
  "risk_score": "low|medium|high|critical",
  "sources": ["url1"]
}
"""
```

`backend/prompts/synthesis_prompt.py`

```
SYNTHESIS_PROMPT = """
You are a Senior Managing Director at Goldman Sachs M&A Advisory.
You have just received research from three specialist analysts on a target company.
Your job: deliver a spoken briefing and write a deal memo.

SPOKEN BRIEFING RULES:
- Maximum 90 seconds of speech (approximately 200-220 words)
- Open with the single most important finding
- Cite specific numbers — never say "strong revenue growth," say "22% YoY revenue growth
- Red flags must be called out explicitly: "Flagging this: [specific issue]"
- End with a clear verdict: BUY / WATCH / AVOID with one-sentence rationale
- Speak in first person as if you've known this company for years
- Do NOT say "based on my research" or "the agents found" — just speak with authority

DEAL MEMO RULES (for Firestore):
Structure as JSON with these exact keys:
{
  "company": "",
  "timestamp": "",
  "session_id": "",
  "verdict": "BUY|WATCH|AVOID",
  "confidence": "high|medium|low",
  "one_liner": "",
  "financials_summary": "",
  "competitive_summary": "",
  "risk_summary": "",
  "red_flags": ["flag1", "flag2"],
  "follow_up_questions": ["q1", "q2", "q3"],
  "spoken_briefing_text": "",
  "sources": ["url1", "url2"]
}
"""
```

`backend/agents/financial_agent.py`

```
from google.adk.agents import LlmAgent
from google.adk.tools import google_search
from backend.prompts.financial_prompt import FINANCIAL_ANALYST_PROMPT


def create_financial_agent() -> LlmAgent:
    """
    Agent A: Researches financials using Google Search grounding.
    Runs in parallel. Writes output to session state key 'financial_data'.
    """
    return LlmAgent(
        name="FinancialAnalyst",
        model="gemini-2.0-flash",
        description="Researches financial metrics, earnings data, and cash flow signals f
        instruction=FINANCIAL_ANALYST_PROMPT,
        tools=[google_search],
        output_key="financial_data",
    )
```

`backend/agents/competitive_agent.py`

```
from google.adk.agents import LlmAgent
from google.adk.tools import google_search
from backend.prompts.competitive_prompt import COMPETITIVE_ANALYST_PROMPT


def create_competitive_agent() -> LlmAgent:
    """
    Agent B: Researches competitive landscape using Google Search grounding.
    Runs in parallel. Writes output to session state key 'competitive_data'.
    """
    return LlmAgent(
        name="CompetitiveAnalyst",
        model="gemini-2.0-flash",
        description="Maps competitive landscape, moat strength, and market threats for M
        instruction=COMPETITIVE_ANALYST_PROMPT,
        tools=[google_search],
        output_key="competitive_data",
    )
```

`backend/agents/sentiment_agent.py`

```
from google.adk.agents import LlmAgent
from google.adk.tools import google_search
from backend.prompts.sentiment_prompt import SENTIMENT_ANALYST_PROMPT


def create_sentiment_agent() -> LlmAgent:
```

```
    """
    Agent C: Researches news sentiment, executive changes, regulatory risk.
    Runs in parallel. Writes output to session state key 'sentiment_data'.
    """
    return LlmAgent(
        name="SentimentAnalyst",
        model="gemini-2.0-flash",
        description="Analyzes news sentiment, executive departures, regulatory risks for
        instruction=SENTIMENT_ANALYST_PROMPT,
        tools=[google_search],
        output_key="sentiment_data",
    )
```

`backend/agents/synthesis_agent.py`

```
from google.adk.agents import LlmAgent
from backend.prompts.synthesis_prompt import SYNTHESIS_PROMPT


def create_synthesis_agent() -> LlmAgent:
    """
    Synthesis Agent: Takes all three agent outputs from session state
    and produces a spoken briefing + structured deal memo JSON.
    Reads from: financial_data, competitive_data, sentiment_data
    Writes to: synthesis_output
    """
    return LlmAgent(
        name="SynthesisAgent",
        model="gemini-2.0-flash",
        description="Synthesizes M&A research into a spoken banker briefing and structur
        instruction=SYNTHESIS_PROMPT
        + """

The financial research is in: {financial_data}
The competitive research is in: {competitive_data}
The sentiment research is in: {sentiment_data}

First output the SPOKEN BRIEFING as plain text (this will be read aloud).
Then output the DEAL MEMO as a JSON block wrapped in ```json ... ```.
""",
        output_key="synthesis_output",
    )
```

`backend/agents/orchestrator.py`

```
import json
import asyncio
from typing import Optional

from google.adk.agents import LlmAgent, ParallelAgent, SequentialAgent
from google.adk.runners import Runner
```

```python
from google.adk.sessions import InMemorySessionService
from google.adk import types

from backend.agents.financial_agent import create_financial_agent
from backend.agents.competitive_agent import create_competitive_agent
from backend.agents.sentiment_agent import create_sentiment_agent
from backend.agents.synthesis_agent import create_synthesis_agent
from backend.services.firestore_service import FirestoreService
from backend.prompts.orchestrator_prompt import ORCHESTRATOR_SYSTEM_PROMPT


class MAOrchestrator:
    """
    Root orchestrator for the M&A War Room.

    Architecture:
    SequentialAgent
      └── ParallelAgent (runs A, B, C simultaneously)
            ├── FinancialAnalyst   (Agent A)
            ├── CompetitiveAnalyst (Agent B)
            └── SentimentAnalyst   (Agent C)
      └── SynthesisAgent (reads A+B+C, produces briefing + memo)
    """

    def __init__(self):
        self.session_service = InMemorySessionService()
        self.firestore = FirestoreService()
        self._build_agent_pipeline()

    def _build_agent_pipeline(self):
        # Three parallel research agents
        financial_agent = create_financial_agent()
        competitive_agent = create_competitive_agent()
        sentiment_agent = create_sentiment_agent()

        # ParallelAgent runs all three simultaneously
        # Each writes to a different output_key in shared session state
        research_parallel = ParallelAgent(
            name="ResearchTeam",
            sub_agents=[financial_agent, competitive_agent, sentiment_agent],
        )

        # Synthesis agent reads all three state keys
        synthesis_agent = create_synthesis_agent()

        # SequentialAgent: first run parallel research, then synthesize
        self.pipeline = SequentialAgent(
            name="MAPipeline",
            sub_agents=[research_parallel, synthesis_agent],
        )

        # Runner executes the pipeline
        self.runner = Runner(
            agent=self.pipeline,
            app_name="ma_warroom",
            session_service=self.session_service,
```

```
        )

    async def analyze_company(
        self,
        company_name: str,
        session_id: str,
        on_status_update: Optional[callable] = None,
    ) -> dict:
        """
        Main entry point. Given a company name, runs full parallel
        research pipeline and returns synthesis output.

        Args:
            company_name: The target company (e.g., "Apple", "Stripe")
            session_id: Unique session ID for this war room session
            on_status_update: Async callback for streaming status to frontend

        Returns:
            dict with 'spoken_briefing' and 'deal_memo' keys
        """
        user_id = "warroom_user"

        # Create session
        await self.session_service.create_session(
            app_name="ma_warroom",
            user_id=user_id,
            session_id=session_id,
        )

        # Inject company name into initial message
        initial_message = f"Analyze {company_name} for M&A due diligence."

        if on_status_update:
            await on_status_update(
                {
                    "type": "agent_status",
                    "agents": [
                        "FinancialAnalyst",
                        "CompetitiveAnalyst",
                        "SentimentAnalyst",
                    ],
                    "status": "running",
                    "company": company_name,
                }
            )

        # Run the pipeline
        final_response_text = ""
        async for event in self.runner.run_async(
            user_id=user_id,
            session_id=session_id,
            new_message=types.Content(
                role="user",
                parts=[types.Part(text=initial_message)],
            ),
        ):
```

```
        # Capture final synthesis output
        if event.is_final_response():
            if event.content and event.content.parts:
                for part in event.content.parts:
                    if part.text:
                        final_response_text += part.text

        # Stream intermediate status updates to frontend
        if on_status_update and hasattr(event, "author"):
            if event.author in [
                "FinancialAnalyst",
                "CompetitiveAnalyst",
                "SentimentAnalyst",
            ]:
                await on_status_update(
                    {"type": "agent_complete", "agent": event.author}
                )

    # Parse synthesis output
    result = self._parse_synthesis_output(final_response_text)
    result["company"] = company_name
    result["session_id"] = session_id

    # Write deal memo to Firestore
    await self.firestore.write_deal_memo(session_id, result["deal_memo"])

    if on_status_update:
        await on_status_update(
            {"type": "pipeline_complete", "session_id": session_id}
        )

    return result

def _parse_synthesis_output(self, raw_text: str) -> dict:
    """
    Parses the synthesis agent output into spoken_briefing + deal_memo.
    The synthesis agent is prompted to output:
    1. Plain text briefing (before the JSON block)
    2. JSON block wrapped in ```json ... ```
    """
    spoken_briefing = raw_text
    deal_memo = {}

    if "```json" in raw_text:
        parts = raw_text.split("```json")
        spoken_briefing = parts[0].strip()
        json_part = parts[1].split("```").strip()
        try:
            deal_memo = json.loads(json_part)
        except json.JSONDecodeError:
            deal_memo = {"raw": json_part, "parse_error": True}

    return {"spoken_briefing": spoken_briefing, "deal_memo": deal_memo}
```

`backend/services/firestore_service.py`

```
import asyncio
from datetime import datetime
from google.cloud import firestore
from google.cloud.firestore_v1.async_client import AsyncClient
import os


class FirestoreService:
    """
    Async Firestore service for writing and reading deal memos.

    Collection structure:
    /deals/{session_id}/
        - company: string
        - timestamp: datetime
        - verdict: BUY|WATCH|AVOID
        - spoken_briefing_text: string
        - red_flags: array
        - financials_summary: string
        - competitive_summary: string
        - risk_summary: string
        - sources: array
        - follow_up_questions: array
    """

    def __init__(self):
        project_id = os.getenv("FIRESTORE_PROJECT_ID")
        self.db = firestore.AsyncClient(project=project_id)

    async def write_deal_memo(self, session_id: str, memo: dict) -> str:
        """Write deal memo to Firestore. Returns document path."""
        doc_ref = self.db.collection("deals").document(session_id)

        memo["timestamp"] = datetime.utcnow()
        memo["session_id"] = session_id

        await doc_ref.set(memo, merge=True)
        return f"deals/{session_id}"

    async def update_field(self, session_id: str, field: str, value) -> None:
        """Update a specific field in an existing deal memo."""
        doc_ref = self.db.collection("deals").document(session_id)
        await doc_ref.update({field: value})

    async def get_deal_memo(self, session_id: str) -> dict:
        """Retrieve a deal memo by session ID."""
        doc_ref = self.db.collection("deals").document(session_id)
        doc = await doc_ref.get()
        if doc.exists:
            return doc.to_dict()
        return {}

    async def append_red_flag(self, session_id: str, flag: str) -> None:
        """Append a red flag to existing deal memo (for live interruptions)."""
```

```
        doc_ref = self.db.collection("deals").document(session_id)
        await doc_ref.update(
            {"red_flags": firestore.ArrayUnion([flag])}
        )

    async def list_recent_deals(self, limit: int = 10) -> list:
        """List most recent deal memos for the session history panel."""
        docs = (
            self.db.collection("deals")
            .order_by("timestamp", direction=firestore.Query.DESCENDING)
            .limit(limit)
            .stream()
        )
        results = []
        async for doc in docs:
            results.append({"id": doc.id, **doc.to_dict()})
        return results
```

backend/services/audio_utils.py

```
import audioop
import struct


def pcm_to_wav_header(sample_rate: int = 16000, channels: int = 1, bits: int = 16) -> byt
    """
    Generate WAV file header for raw PCM audio.
    Gemini Live API expects LINEAR16 PCM at 16kHz mono.
    """
    byte_rate = sample_rate * channels * bits // 8
    block_align = channels * bits // 8

    # This is a placeholder WAV header —
    # actual data_size filled when streaming ends
    header = struct.pack(
        '<4sI4s4sIHHIIHH4sI',
        b'RIFF', 0,  # total size placeholder
        b'WAVE',
        b'fmt ', 16,
        1,          # PCM format
        channels,
        sample_rate,
        byte_rate,
        block_align,
        bits,
        b'data', 0   # data size placeholder
    )
    return header


def convert_browser_audio_to_pcm(raw_audio: bytes) -> bytes:
    """
    Browser MediaRecorder typically outputs WebM/Opus or PCM.
    Gemini Live API needs LINEAR16 PCM at 16000Hz.
```

```
    For hackathon: assume browser sends raw PCM at 16kHz.
    For production: use ffmpeg or soundfile for conversion.
    """
    return raw_audio


def chunk_audio(audio_bytes: bytes, chunk_size_ms: int = 100, sample_rate: int = 16000) -
    """
    Split audio into chunks for streaming to Gemini Live API.
    chunk_size_ms=100 means 100ms chunks.
    bytes_per_chunk = sample_rate * (ms/1000) * 2 bytes (16-bit) * 1 channel
    """
    bytes_per_chunk = int(sample_rate * (chunk_size_ms / 1000) * 2)
    return [
        audio_bytes[i : i + bytes_per_chunk]
        for i in range(0, len(audio_bytes), bytes_per_chunk)
    ]
```

backend/live_session.py

```
import asyncio
import json
import os
from typing import AsyncGenerator, Optional, Callable

from google import genai
from google.genai import types as genai_types


class LiveSession:
    """
    Manages a single Gemini Live API WebSocket session.

    Handles:
    - Bidirectional audio streaming (user mic → Gemini → speaker)
    - Barge-in / interruption detection (Gemini handles natively)
    - Company name intent detection from live audio
    - Turn completion events → triggers ADK pipeline

    Gemini Live API uses:
    - Model: gemini-2.0-flash-live-001
    - Audio in: LINEAR16 PCM 16000Hz
    - Audio out: LINEAR16 PCM 24000Hz (Gemini output)
    - Text out: for intent parsing
    """

    LIVE_MODEL = "gemini-2.0-flash-live-001"

    LIVE_SYSTEM_PROMPT = """
You are the voice interface for an M&A Due Diligence War Room.

Your role:
1. Listen for the user speaking a company name or saying "analyze [company]"
```

```
2. Confirm you heard it: "Analyzing [Company]. Spinning up research team now."
3. When research results arrive (injected as text), read the banker briefing aloud
4. Handle follow-up questions conversationally, drawing from the loaded context
5. If user says "stop" or "pause", stop speaking immediately
6. If user says "what are the red flags?", enumerate them specifically
7. If user says "give me the memo", confirm it's saved and give the Firestore reference

Tone: Senior investment banker. Crisp, authoritative, numbers-first.
Never say "I don't know" — if data is missing, say "Data unavailable — flag for manual ch
"""

    def __init__(self, session_id: str):
        self.session_id = session_id
        self.client = genai.Client(api_key=os.getenv("GOOGLE_API_KEY"))
        self._session = None
        self._company_detected_callback: Optional[Callable] = None
        self._audio_output_callback: Optional[Callable] = None
        self._text_output_callback: Optional[Callable] = None

    def on_company_detected(self, callback: Callable):
        """Register callback for when a company name is detected."""
        self._company_detected_callback = callback

    def on_audio_output(self, callback: Callable):
        """Register callback for outgoing audio chunks (to frontend)."""
        self._audio_output_callback = callback

    def on_text_output(self, callback: Callable):
        """Register callback for text transcript (for frontend feed)."""
        self._text_output_callback = callback

    async def start(self):
        """Initialize the Live API session."""
        config = genai_types.LiveConnectConfig(
            response_modalities=["AUDIO", "TEXT"],
            speech_config=genai_types.SpeechConfig(
                voice_config=genai_types.VoiceConfig(
                    prebuilt_voice_config=genai_types.PrebuiltVoiceConfig(
                        voice_name="Charon"  # Deep, authoritative voice
                    )
                )
            ),
            system_instruction=genai_types.Content(
                parts=[genai_types.Part(text=self.LIVE_SYSTEM_PROMPT)]
            ),
        )

        self._session = await self.client.aio.live.connect(
            model=self.LIVE_MODEL,
            config=config,
        )

        # Start receiving responses in background
        asyncio.create_task(self._receive_loop())

    async def send_audio(self, audio_chunk: bytes):
```

```
        """
        Send raw PCM audio chunk to Gemini Live API.
        Called continuously from WebSocket handler as mic data arrives.
        """
        if self._session:
            await self._session.send(
                input=genai_types.LiveClientRealtimeInput(
                    media_chunks=[
                        genai_types.Blob(
                            data=audio_chunk,
                            mime_type="audio/pcm;rate=16000",
                        )
                    ]
                )
            )

    async def inject_briefing(self, briefing_text: str, company: str):
        """
        After ADK pipeline completes, inject the briefing as a text message
        so Live API reads it aloud in the banker's voice.
        """
        if self._session:
            injection = f"""
The research team has completed their analysis.
Deliver the following briefing for {company}:

{briefing_text}
"""
            await self._session.send(
                input=genai_types.LiveClientContentUpdate(
                    turns=[
                        genai_types.LiveClientTurn(
                            role="user",
                            parts=[genai_types.Part(text=injection)],
                        )
                    ],
                    turn_complete=True,
                )
            )

    async def send_text(self, text: str):
        """Send a text message (for follow-up questions typed or injected)."""
        if self._session:
            await self._session.send(
                input=genai_types.LiveClientContentUpdate(
                    turns=[
                        genai_types.LiveClientTurn(
                            role="user",
                            parts=[genai_types.Part(text=text)],
                        )
                    ],
                    turn_complete=True,
                )
            )

    async def _receive_loop(self):
```

```
        """
        Continuously receive responses from Gemini Live API.
        Handles: audio output, text output, company detection, interruptions.
        """
        if not self._session:
            return

        async for response in self._session.receive():
            # Handle audio output chunks
            if response.data:
                if self._audio_output_callback:
                    await self._audio_output_callback(response.data)

            # Handle text output (transcript)
            if response.text:
                if self._text_output_callback:
                    await self._text_output_callback(response.text)

                # Check if this text contains a company detection confirmation
                # Live API will say "Analyzing [Company]..." per system prompt
                await self._check_for_company_intent(response.text)

            # Handle turn completion
            if response.server_content:
                if hasattr(response.server_content, "turn_complete"):
                    if response.server_content.turn_complete:
                        pass  # Turn complete — can trigger next action

                # Handle interruption (barge-in)
                if hasattr(response.server_content, "interrupted"):
                    if response.server_content.interrupted:
                        if self._text_output_callback:
                            await self._text_output_callback(
                                "[INTERRUPTED — listening...]"
                            )

    async def _check_for_company_intent(self, text: str):
        """
        Detect if Gemini confirmed it heard a company name.
        Triggers ADK pipeline callback.

        Heuristic: Live API system prompt instructs it to say
        "Analyzing [Company]. Spinning up research team now."
        We parse that confirmation.
        """
        lower = text.lower()
        if "analyzing" in lower and "spinning up" in lower:
            # Extract company name from the confirmation phrase
            words = text.split()
            try:
                analyzing_idx = [w.lower() for w in words].index("analyzing")
                company_name = words[analyzing_idx + 1].rstrip(".,")
                if self._company_detected_callback:
                    await self._company_detected_callback(company_name)
            except (ValueError, IndexError):
                pass
```

```
    async def close(self):
        """Close the Live API session gracefully."""
        if self._session:
            await self._session.close()
            self._session = None
```

backend/main.py

```
import asyncio
import json
import os
import uuid
from contextlib import asynccontextmanager
from typing import Dict

import uvicorn
from dotenv import load_dotenv
from fastapi import FastAPI, WebSocket, WebSocketDisconnect
from fastapi.middleware.cors import CORSMiddleware

from backend.live_session import LiveSession
from backend.agents.orchestrator import MAOrchestrator

load_dotenv()

# Active sessions: session_id → {live_session, orchestrator, websocket}
active_sessions: Dict[str, dict] = {}


@asynccontextmanager
async def lifespan(app: FastAPI):
    print("M&A War Room backend starting up...")
    yield
    print("Shutting down — closing all sessions...")
    for sid, session_data in active_sessions.items():
        await session_data["live_session"].close()


app = FastAPI(
    title="M&A War Room API",
    description="Live M&A Due Diligence Agent — Google x Columbia Hackathon 2026",
    lifespan=lifespan,
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],  # Tighten for production
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

```
@app.get("/health")
async def health():
    return {"status": "war room ready", "active_sessions": len(active_sessions)}


@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
    """
    Main WebSocket endpoint.

    Message types FROM client (frontend → backend):
    - {"type": "start_session"}          → Initialize new war room session
    - {"type": "audio", "data": <base64>} → Raw PCM audio chunk from mic
    - {"type": "text", "text": "..."}     → Text message (follow-up questions)
    - {"type": "end_session"}            → Close session gracefully

    Message types TO client (backend → frontend):
    - {"type": "session_id", "session_id": "..."}     → Session created
    - {"type": "audio", "data": <base64>}            → Audio to play
    - {"type": "transcript", "text": "..."}          → Live transcript
    - {"type": "agent_status", "agents": [...]}      → Research agents fired
    - {"type": "agent_complete", "agent": "..."}     → Sub-agent finished
    - {"type": "pipeline_complete", "session_id": "..."} → Full analysis done
    - {"type": "deal_memo", "memo": {...}}           → Final structured memo
    - {"type": "red_flag", "flag": "..."}            → Red flag callout
    - {"type": "error", "message": "..."}            → Error event
    """
    await websocket.accept()
    session_id = str(uuid.uuid4())

    # Initialize orchestrator and live session
    orchestrator = MAOrchestrator()
    live_session = LiveSession(session_id=session_id)

    # Register callbacks that push data back through the WebSocket

    async def send_ws(data: dict):
        try:
            await websocket.send_json(data)
        except Exception:
            pass  # Client disconnected

    async def on_audio_output(audio_bytes: bytes):
        """Stream audio back to frontend for playback."""
        import base64
        await send_ws({
            "type": "audio",
            "data": base64.b64encode(audio_bytes).decode(),
        })

    async def on_text_output(text: str):
        """Stream transcript text to frontend feed."""
        await send_ws({"type": "transcript", "text": text})

        # Check for red flag keywords in Live API output
        red_flag_keywords = [
```

```
        "flagging this", "material omission", "discrepancy",
        "going concern", "sec investigation", "cfo departure",
        "guidance cut", "insider selling"
    ]
    lower_text = text.lower()
    for keyword in red_flag_keywords:
        if keyword in lower_text:
            # Extract surrounding context
            idx = lower_text.index(keyword)
            flag_context = text[max(0, idx-20):idx+100]
            await send_ws({"type": "red_flag", "flag": flag_context})

async def on_company_detected(company_name: str):
    """
    Fires when Live API confirms it heard a company name.
    Triggers the ADK parallel research pipeline.
    """
    await send_ws({
        "type": "company_detected",
        "company": company_name,
    })

    async def on_status_update(status: dict):
        await send_ws(status)

    try:
        result = await orchestrator.analyze_company(
            company_name=company_name,
            session_id=session_id,
            on_status_update=on_status_update,
        )

        # Send deal memo to frontend
        await send_ws({
            "type": "deal_memo",
            "memo": result["deal_memo"],
        })

        # Inject spoken briefing back into Live API → plays aloud
        await live_session.inject_briefing(
            briefing_text=result["spoken_briefing"],
            company=company_name,
        )

    except Exception as e:
        await send_ws({"type": "error", "message": str(e)})

# Wire up callbacks
live_session.on_audio_output(on_audio_output)
live_session.on_text_output(on_text_output)
live_session.on_company_detected(on_company_detected)

# Start the Live API session
await live_session.start()
active_sessions[session_id] = {
    "live_session": live_session,
```

```
        "orchestrator": orchestrator,
        "websocket": websocket,
    }

    # Send session ID to frontend
    await send_ws({"type": "session_id", "session_id": session_id})

    # Main message loop
    try:
        while True:
            message = await websocket.receive()

            if "text" in message:
                data = json.loads(message["text"])
                msg_type = data.get("type")

                if msg_type == "text":
                    # Forward text message to Live API
                    await live_session.send_text(data.get("text", ""))

                elif msg_type == "end_session":
                    break

            elif "bytes" in message:
                # Raw binary audio from frontend mic
                await live_session.send_audio(message["bytes"])

    except WebSocketDisconnect:
        pass
    finally:
        await live_session.close()
        active_sessions.pop(session_id, None)


if __name__ == "__main__":
    uvicorn.run(
        "backend.main:app",
        host="0.0.0.0",
        port=int(os.getenv("PORT", 8000)),
        reload=os.getenv("ENVIRONMENT") == "development",
    )
```

`backend/Dockerfile`

```
FROM python:3.11-slim

WORKDIR /app

# Install system dependencies
RUN apt-get update && apt-get install -y \
    gcc \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8080

CMD ["uvicorn", "backend.main:app", "--host", "0.0.0.0", "--port", "8080"]
```

PART 5: FRONTEND — COMPLETE CODE

`frontend/hooks/useWarRoom.ts`

```
import { useState, useRef, useCallback, useEffect } from "react";

export type AgentStatus = "idle" | "running" | "complete" | "error";
export type Verdict = "BUY" | "WATCH" | "AVOID" | null;

export interface DealMemo {
  company: string;
  verdict: Verdict;
  confidence: string;
  one_liner: string;
  financials_summary: string;
  competitive_summary: string;
  risk_summary: string;
  red_flags: string[];
  follow_up_questions: string[];
  sources: string[];
  session_id: string;
}

export interface WarRoomState {
  sessionId: string | null;
  isConnected: boolean;
  isListening: boolean;
  isAnalyzing: boolean;
  currentCompany: string | null;
  transcript: string[];
  redFlags: string[];
  agentStatuses: Record<string, AgentStatus>;
  dealMemo: DealMemo | null;
  audioQueue: ArrayBuffer[];
  error: string | null;
}

const WS_URL =
  process.env.NEXT_PUBLIC_WS_URL || "ws://localhost:8000/ws";

export function useWarRoom() {
  const [state, setState] = useState<WarRoomState>({
    sessionId: null,
    isConnected: false,
    isListening: false,
```

```
    isAnalyzing: false,
    currentCompany: null,
    transcript: [],
    redFlags: [],
    agentStatuses: {
      FinancialAnalyst: "idle",
      CompetitiveAnalyst: "idle",
      SentimentAnalyst: "idle",
      SynthesisAgent: "idle",
    },
    dealMemo: null,
    audioQueue: [],
    error: null,
  });

  const wsRef = useRef<WebSocket | null>(null);
  const mediaRecorderRef = useRef<MediaRecorder | null>(null);
  const audioContextRef = useRef<AudioContext | null>(null);
  const audioWorkletRef = useRef<AudioWorkletNode | null>(null);
  const streamRef = useRef<MediaStream | null>(null);

  // Audio playback queue
  const isPlayingRef = useRef(false);
  const playbackQueueRef = useRef<ArrayBuffer[]>([]);

  const updateState = useCallback((updates: Partial<WarRoomState>) => {
    setState((prev) => ({ ...prev, ...updates }));
  }, []);

  // Play audio from Gemini Live API
  const playAudioChunk = useCallback(async (audioData: ArrayBuffer) => {
    playbackQueueRef.current.push(audioData);
    if (!isPlayingRef.current) {
      isPlayingRef.current = true;
      while (playbackQueueRef.current.length > 0) {
        const chunk = playbackQueueRef.current.shift()!;
        await playPCMChunk(chunk);
      }
      isPlayingRef.current = false;
    }
  }, []);

  const playPCMChunk = async (audioData: ArrayBuffer) => {
    // Gemini Live API outputs LINEAR16 PCM at 24000Hz
    const ctx = audioContextRef.current || new AudioContext({ sampleRate: 24000 });
    audioContextRef.current = ctx;

    const pcmData = new Int16Array(audioData);
    const float32Data = new Float32Array(pcmData.length);
    for (let i = 0; i < pcmData.length; i++) {
      float32Data[i] = pcmData[i] / 32768.0;
    }

    const audioBuffer = ctx.createBuffer(1, float32Data.length, 24000);
    audioBuffer.copyToChannel(float32Data, 0);
```

```
    const source = ctx.createBufferSource();
    source.buffer = audioBuffer;
    source.connect(ctx.destination);

    return new Promise<void>((resolve) => {
      source.onended = () => resolve();
      source.start();
    });
  };

  // Connect WebSocket
  const connect = useCallback(() => {
    const ws = new WebSocket(WS_URL);

    ws.onopen = () => {
      updateState({ isConnected: true });
    };

    ws.onmessage = async (event) => {
      const data = JSON.parse(event.data);

      switch (data.type) {
        case "session_id":
          updateState({ sessionId: data.session_id });
          break;

        case "audio":
          // Decode base64 audio and play
          const audioBytes = Uint8Array.from(atob(data.data), (c) =>
            c.charCodeAt(0)
          ).buffer;
          await playAudioChunk(audioBytes);
          break;

        case "transcript":
          setState((prev) => ({
            ...prev,
            transcript: [...prev.transcript.slice(-50), data.text],
          }));
          break;

        case "company_detected":
          updateState({
            currentCompany: data.company,
            isAnalyzing: true,
          });
          break;

        case "agent_status":
          const agentStatuses: Record<string, AgentStatus> = {};
          data.agents.forEach((a: string) => {
            agentStatuses[a] = "running";
          });
          setState((prev) => ({
            ...prev,
            agentStatuses: { ...prev.agentStatuses, ...agentStatuses },
```

```
        }));
        break;

      case "agent_complete":
        setState((prev) => ({
          ...prev,
          agentStatuses: {
            ...prev.agentStatuses,
            [data.agent]: "complete",
          },
        }));
        break;

      case "pipeline_complete":
        updateState({ isAnalyzing: false });
        setState((prev) => ({
          ...prev,
          agentStatuses: {
            FinancialAnalyst: "complete",
            CompetitiveAnalyst: "complete",
            SentimentAnalyst: "complete",
            SynthesisAgent: "complete",
          },
        }));
        break;

      case "deal_memo":
        updateState({ dealMemo: data.memo });
        break;

      case "red_flag":
        setState((prev) => ({
          ...prev,
          redFlags: [...prev.redFlags, data.flag],
        }));
        break;

      case "error":
        updateState({ error: data.message, isAnalyzing: false });
        break;
    }
  };

  ws.onclose = () => {
    updateState({ isConnected: false, isListening: false });
  };

  ws.onerror = (err) => {
    updateState({ error: "Connection failed. Check backend." });
  };

  wsRef.current = ws;
}, [updateState, playAudioChunk]);

// Start microphone
const startListening = useCallback(async () => {
```

```
    if (!wsRef.current || wsRef.current.readyState !== WebSocket.OPEN) {
      connect();
      await new Promise((r) => setTimeout(r, 500));
    }

    try {
      const stream = await navigator.mediaDevices.getUserMedia({
        audio: {
          sampleRate: 16000,
          channelCount: 1,
          echoCancellation: true,
          noiseSuppression: true,
        },
      });

      streamRef.current = stream;

      // Use ScriptProcessorNode for raw PCM (simpler for hackathon)
      const ctx = new AudioContext({ sampleRate: 16000 });
      audioContextRef.current = ctx;
      const source = ctx.createMediaStreamSource(stream);
      const processor = ctx.createScriptProcessor(4096, 1, 1);

      processor.onaudioprocess = (e) => {
        const inputData = e.inputBuffer.getChannelData(0);
        // Convert Float32 to Int16 PCM
        const pcm = new Int16Array(inputData.length);
        for (let i = 0; i < inputData.length; i++) {
          const s = Math.max(-1, Math.min(1, inputData[i]));
          pcm[i] = s < 0 ? s * 0x8000 : s * 0x7fff;
        }
        // Send raw binary PCM to backend
        if (wsRef.current?.readyState === WebSocket.OPEN) {
          wsRef.current.send(pcm.buffer);
        }
      };

      source.connect(processor);
      processor.connect(ctx.destination);
      updateState({ isListening: true });
    } catch (err) {
      updateState({ error: "Microphone access denied." });
    }
  }, [connect, updateState]);

  // Stop microphone
  const stopListening = useCallback(() => {
    streamRef.current?.getTracks().forEach((t) => t.stop());
    updateState({ isListening: false });
  }, [updateState]);

  // Send text follow-up
  const sendFollowUp = useCallback((text: string) => {
    wsRef.current?.send(JSON.stringify({ type: "text", text }));
  }, []);
```

```
  // Disconnect
  const disconnect = useCallback(() => {
    wsRef.current?.send(JSON.stringify({ type: "end_session" }));
    wsRef.current?.close();
    streamRef.current?.getTracks().forEach((t) => t.stop());
    updateState({ isConnected: false, isListening: false });
  }, [updateState]);

  // Auto-connect on mount
  useEffect(() => {
    connect();
    return () => {
      wsRef.current?.close();
    };
  }, []);

  return {
    state,
    startListening,
    stopListening,
    sendFollowUp,
    disconnect,
  };
}
```

`frontend/app/page.tsx`

```
"use client";

import { useWarRoom } from "@/hooks/useWarRoom";
import { MicButton } from "@/components/MicButton";
import { BriefingFeed } from "@/components/BriefingFeed";
import { DealMemoPanel } from "@/components/DealMemo";
import { AgentStatusPanel } from "@/components/AgentStatus";
import { RedFlagAlert } from "@/components/RedFlagAlert";

export default function WarRoomPage() {
  const { state, startListening, stopListening, sendFollowUp } = useWarRoom();

  const verdictColor = {
    BUY: "text-green-400",
    WATCH: "text-yellow-400",
    AVOID: "text-red-400",
    null: "text-gray-400",
  }[state.dealMemo?.verdict ?? "null"];

  return (
    <main className="min-h-screen bg-gray-950 text-white font-mono p-6">
      {/* Header */}
      <div className="flex items-center justify-between mb-8">
        <div>
          <h1 className="text-2xl font-bold tracking-tight">
            □ M&A WAR ROOM
          </h1>
```

```
          <p className="text-gray-400 text-sm">
            Google x Columbia Hackathon 2026 — Live Agent Track
          </p>
        </div>
        <div className="flex items-center gap-3">
          <div
            className={`w-2 h-2 rounded-full ${
              state.isConnected ? "bg-green-400 animate-pulse" : "bg-red-500"
            }`}
          />
          <span className="text-xs text-gray-400">
            {state.isConnected ? "CONNECTED" : "DISCONNECTED"}
          </span>
        </div>
      </div>

      {/* Red flag alerts */}
      {state.redFlags.slice(-3).map((flag, i) => (
        <RedFlagAlert key={i} flag={flag} />
      ))}

      {/* Main grid */}
      <div className="grid grid-cols-3 gap-6">
        {/* Left: Mic + Agent Status */}
        <div className="col-span-1 space-y-6">
          <div className="bg-gray-900 rounded-xl p-6 flex flex-col items-center gap-4">
            <p className="text-xs text-gray-400 uppercase tracking-widest">
              {state.isListening
                ? "LISTENING — SAY A COMPANY NAME"
                : "CLICK TO ACTIVATE WAR ROOM"}
            </p>
            <MicButton
              isListening={state.isListening}
              isAnalyzing={state.isAnalyzing}
              onStart={startListening}
              onStop={stopListening}
            />
            {state.currentCompany && (
              <div className="text-center">
                <p className="text-gray-400 text-xs">ANALYZING</p>
                <p className="text-xl font-bold text-blue-400">
                  {state.currentCompany.toUpperCase()}
                </p>
              </div>
            )}
          </div>

          <AgentStatusPanel statuses={state.agentStatuses} />

          {/* Follow-up input */}
          {state.dealMemo && (
            <div className="bg-gray-900 rounded-xl p-4">
              <p className="text-xs text-gray-400 mb-2">FOLLOW-UP QUESTION</p>
              <div className="flex gap-2">
                <input
                  className="flex-1 bg-gray-800 rounded px-3 py-2 text-sm outline-none"
```

```
                placeholder="What's the EBITDA margin?"
                onKeyDown={(e) => {
                  if (e.key === "Enter") {
                    sendFollowUp((e.target as HTMLInputElement).value);
                    (e.target as HTMLInputElement).value = "";
                  }
                }}
              />
            </div>
            {/* Suggested follow-ups from deal memo */}
            <div className="mt-3 flex flex-wrap gap-2">
              {state.dealMemo.follow_up_questions?.slice(0, 3).map((q, i) => (
                <button
                  key={i}
                  onClick={() => sendFollowUp(q)}
                  className="text-xs bg-gray-800 hover:bg-gray-700 rounded px-2 py-1 te
                >
                  {q}
                </button>
              ))}
            </div>
          </div>
        )}
      </div>

      {/* Center: Live briefing transcript */}
      <div className="col-span-1">
        <BriefingFeed transcript={state.transcript} />
      </div>

      {/* Right: Deal memo */}
      <div className="col-span-1">
        {state.dealMemo ? (
          <DealMemoPanel memo={state.dealMemo} verdictColor={verdictColor} />
        ) : (
          <div className="bg-gray-900 rounded-xl p-6 h-full flex items-center justify-c
            <p className="text-gray-600 text-sm text-center">
              Deal memo appears here after analysis
            </p>
          </div>
        )}
      </div>
    </div>
  </main>
);
}
```

frontend/components/AgentStatus.tsx

```
import { AgentStatus } from "@/hooks/useWarRoom";

const agents = [
  { key: "FinancialAnalyst", label: "Financial Analyst", icon: "□" },
  { key: "CompetitiveAnalyst", label: "Competitive Intel", icon: "□" },
```

```
  { key: "SentimentAnalyst", label: "Sentiment & Risk", icon: "□" },
  { key: "SynthesisAgent", label: "Synthesis MD", icon: "□" },
];

const statusStyles: Record<AgentStatus, string> = {
  idle: "bg-gray-700 text-gray-400",
  running: "bg-blue-900 text-blue-300 animate-pulse",
  complete: "bg-green-900 text-green-400",
  error: "bg-red-900 text-red-400",
};

const statusLabel: Record<AgentStatus, string> = {
  idle: "STANDBY",
  running: "RUNNING",
  complete: "DONE",
  error: "ERROR",
};

export function AgentStatusPanel({
  statuses,
}: {
  statuses: Record<string, AgentStatus>;
}) {
  return (
    <div className="bg-gray-900 rounded-xl p-4">
      <p className="text-xs text-gray-400 uppercase tracking-widest mb-3">
        Research Team
      </p>
      <div className="space-y-2">
        {agents.map(({ key, label, icon }) => (
          <div
            key={key}
            className={`flex items-center justify-between rounded-lg px-3 py-2 ${
              statusStyles[statuses[key] ?? "idle"]
            }`}
          >
            <span className="text-sm">
              {icon} {label}
            </span>
            <span className="text-xs font-bold">
              {statusLabel[statuses[key] ?? "idle"]}
            </span>
          </div>
        ))}
      </div>
    </div>
  );
}
```

`frontend/components/RedFlagAlert.tsx`

```
import { useEffect, useState } from "react";

export function RedFlagAlert({ flag }: { flag: string }) {
  const [visible, setVisible] = useState(true);

  useEffect(() => {
    const t = setTimeout(() => setVisible(false), 6000);
    return () => clearTimeout(t);
  }, []);

  if (!visible) return null;

  return (
    <div className="bg-red-950 border border-red-500 rounded-lg px-4 py-3 mb-3 flex ite
      <span className="text-red-400 text-lg">⚠</span>
      <div>
        <p className="text-red-400 text-xs font-bold uppercase tracking-widest">
          Red Flag Detected
        </p>
        <p className="text-red-200 text-sm mt-1">{flag}</p>
      </div>
    </div>
  );
}
```

PART 6: FIRESTORE CONFIGURATION

`firestore.rules`

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /deals/{sessionId} {
      allow read, write: if true; // Hackathon — lock down for production
    }
  }
}
```

Firestore Index Setup (run once)

```
# Create compound index for listing recent deals
gcloud firestore indexes composite create \
  --collection-group=deals \
  --field-config field-path=timestamp,order=DESCENDING \
  --project=YOUR_PROJECT_ID
```

PART 7: DEPLOYMENT

Cloud Run Deployment (Backend)

```
# 1. Build and push Docker image
gcloud builds submit --tag gcr.io/YOUR_PROJECT_ID/ma-warroom-backend

# 2. Deploy to Cloud Run
gcloud run deploy ma-warroom-backend \
  --image gcr.io/YOUR_PROJECT_ID/ma-warroom-backend \
  --platform managed \
  --region us-central1 \
  --allow-unauthenticated \
  --set-env-vars GOOGLE_API_KEY=YOUR_KEY,FIRESTORE_PROJECT_ID=YOUR_PROJECT_ID \
  --memory 2Gi \
  --cpu 2 \
  --timeout 3600 \
  --concurrency 10

# 3. Get URL
gcloud run services describe ma-warroom-backend \
  --platform managed \
  --region us-central1 \
  --format 'value(status.url)'
```

Firebase Hosting (Frontend)

```
# In /frontend
npm run build
firebase deploy --only hosting
```

`firebase.json`

```
{
  "hosting": {
    "public": "frontend/out",
    "ignore": ["firebase.json", "**/.*", "**/node_modules/**"],
    "rewrites": [{ "source": "**", "destination": "/index.html" }]
  }
}
```

PART 8: 6-HOUR BUILD TIMELINE

```
HOUR 0:00-0:30 → SETUP (Do this before hackathon starts)
  ✅ GCP project created + billing enabled
  ✅ Firestore database initialized (Native mode, us-central1)
  ✅ Google AI Studio API key generated
  ✅ Firebase project linked
```

```
  ✅ Python venv created + requirements.txt installed
  ✅ Next.js project initialized
  ✅ All .env files populated

HOUR 0:30-1:30 → ADK AGENTS (Core intelligence)
  [ ] Write all 5 prompt files (orchestrator, financial, competitive, sentiment, synthes
  [ ] Write financial_agent.py, competitive_agent.py, sentiment_agent.py
  [ ] Write synthesis_agent.py
  [ ] Write orchestrator.py with ParallelAgent + SequentialAgent
  [ ] TEST: Run orchestrator.analyze_company("Apple", "test-001") from CLI
  [ ] Verify three sub-agents fire in parallel + Firestore write works
  MILESTONE: "analyze Apple" → deal memo in Firestore ✅

HOUR 1:30-2:30 → LIVE API SESSION (Voice layer)
  [ ] Write live_session.py
  [ ] Write audio_utils.py
  [ ] TEST: Run LiveSession standalone, speak into mic, get audio back
  [ ] Wire on_company_detected callback → logs company name
  MILESTONE: Speak "analyze Stripe" → console logs "Company detected: Stripe" ✅

HOUR 2:30-3:30 → FASTAPI WEBSOCKET (Glue layer)
  [ ] Write main.py with /ws endpoint
  [ ] Wire LiveSession + MAOrchestrator together
  [ ] Write Firestore service
  [ ] TEST: wscat ws://localhost:8000/ws → send audio → receive session_id
  [ ] TEST: Verify full pipeline end-to-end: voice in → memo in Firestore
  MILESTONE: Full backend pipeline working locally ✅

HOUR 3:30-4:30 → FRONTEND (UI + WebSocket client)
  [ ] Write useWarRoom.ts hook
  [ ] Write page.tsx war room UI
  [ ] Write AgentStatus, BriefingFeed, DealMemo, RedFlagAlert components
  [ ] TEST: Connect frontend to local backend → see agent status update
  [ ] TEST: Speak company name → see transcript + red flag alerts
  MILESTONE: Full UI working against local backend ✅

HOUR 4:30-5:15 → DEPLOY
  [ ] Docker build + Cloud Run deploy backend
  [ ] Update frontend .env.local with Cloud Run WebSocket URL
  [ ] Firebase deploy frontend
  [ ] TEST: Full demo flow on deployed URLs
  MILESTONE: Deployed and working ✅

HOUR 5:15-6:00 → DEMO PREP + POLISH
  [ ] Rehearse 3-minute demo script (below)
  [ ] Pick 3 companies with interesting dynamics (Tesla, OpenAI, WeWork 2.0)
  [ ] Verify red flags trigger on at least one company
  [ ] Add loading states, smooth animations
  [ ] Screenshot Firestore deal memo for slides
  [ ] Prepare 1-slide architecture diagram for judging
```

PART 9: DEMO SCRIPT (3 Minutes)

```
[0:00] "This is the M&A War Room. I'm going to speak one company name.
        That's it. Watch what happens."

[0:10] [Click mic, say "analyze Tesla"]

[0:12] Live API responds: "Analyzing Tesla. Spinning up research team now."
       → Three agent cards light up GREEN on screen: Financial / Competitive / Sentiment

[0:30] First agent completes → second → third
       → Firestore write triggers → Deal Memo panel fills in real-time

[0:45] Live API begins spoken briefing in banker voice:
       "Tesla's revenue growth decelerated to 8% last quarter — well below
        the 15% consensus estimate. EBITDA margins are compressing.
        Flagging this: operating cash flow dropped 22% while Elon Musk
        cited 'macro headwinds' — but competitor BYD posted 31% growth
        in the same period. Competitive moat is weakening.
        Risk signals: three Wall Street downgrades in the last 30 days,
        and we're seeing unusual insider selling patterns.
        Verdict: WATCH — not a buy until margin trajectory clarifies."

[1:30] [Interrupt mid-briefing] "What's their debt situation?"
       → Agent answers immediately from loaded context
       "Tesla carries approximately $5.8B net debt, up from $2.1B prior year.
        Interest coverage is still healthy but trending in the wrong direction."

[2:00] Show Firestore console: deal memo document with full structured data
       "Everything you just heard — saved to Firestore in real-time.
        Every red flag. Every data point. Shareable, queryable, auditable."

[2:30] "This replaces $25,000/year Bloomberg Terminal subscriptions
        for buy-side analysts at hedge funds and boutique M&A shops.
        Built on Gemini Live API, ADK ParallelAgent, Google Search Grounding,
        and Firestore. Deployed on Cloud Run. Built in 6 hours."

[2:50] Open Q&A — follow-up questions panel is live for judges to ask
```

PART 10: TEST CASES

Unit Tests — Run Before Submitting

```
# test_agents.py — run with: python -m pytest test_agents.py -v

import asyncio
import pytest
from backend.agents.orchestrator import MAOrchestrator


@pytest.mark.asyncio
async def test_full_pipeline_apple():
```

```
    """
    Core integration test: full pipeline for Apple Inc.
    Should return spoken briefing + structured deal memo.
    """
    orchestrator = MAOrchestrator()
    result = await orchestrator.analyze_company(
        company_name="Apple",
        session_id="test-apple-001",
    )

    assert "spoken_briefing" in result
    assert len(result["spoken_briefing"]) > 100  # Non-empty briefing

    memo = result["deal_memo"]
    assert "verdict" in memo
    assert memo["verdict"] in ["BUY", "WATCH", "AVOID"]
    assert "red_flags" in memo
    assert isinstance(memo["red_flags"], list)
    assert "financials_summary" in memo
    assert "competitive_summary" in memo


@pytest.mark.asyncio
async def test_parallel_execution_speed():
    """
    Verify that ParallelAgent runs 3 sub-agents in parallel, not sequentially.
    Should complete in under 30 seconds (sequential would take 60-90s).
    """
    import time
    orchestrator = MAOrchestrator()

    start = time.time()
    result = await orchestrator.analyze_company(
        company_name="Microsoft",
        session_id="test-speed-001",
    )
    elapsed = time.time() - start

    assert elapsed < 45, f"Pipeline took {elapsed:.1f}s — check ParallelAgent"
    assert result["spoken_briefing"] != ""


@pytest.mark.asyncio
async def test_firestore_write():
    """
    Verify deal memo is written to Firestore correctly.
    """
    from backend.services.firestore_service import FirestoreService

    service = FirestoreService()
    test_memo = {
        "company": "TestCo",
        "verdict": "WATCH",
        "red_flags": ["Test flag 1"],
        "financials_summary": "Test financials",
        "competitive_summary": "Test competitive",
```

```
        "risk_summary": "Test risk",
    }

    session_id = "test-firestore-write-001"
    path = await service.write_deal_memo(session_id, test_memo)
    assert path == f"deals/{session_id}"

    retrieved = await service.get_deal_memo(session_id)
    assert retrieved["company"] == "TestCo"
    assert retrieved["verdict"] == "WATCH"


@pytest.mark.asyncio
async def test_red_flag_detection_high_risk_company():
    """
    Test that a high-risk company triggers at least one red flag.
    Use a historically troubled company to validate flag detection.
    """
    orchestrator = MAOrchestrator()
    result = await orchestrator.analyze_company(
        company_name="WeWork",
        session_id="test-redflag-001",
    )

    memo = result["deal_memo"]
    # WeWork should have red flags given its history
    assert len(memo.get("red_flags", [])) > 0, "High-risk company should trigger flags"
    assert memo.get("verdict") in ["WATCH", "AVOID"]


def test_synthesis_output_parsing():
    """
    Unit test for the synthesis output parser.
    Validates JSON extraction from mixed text+JSON output.
    """
    from backend.agents.orchestrator import MAOrchestrator

    orchestrator = MAOrchestrator()
    test_output = """
Apple shows strong revenue growth of 8% YoY. Competitive moat remains strong via ecosys

```json
{
  "company": "Apple",
  "verdict": "BUY",
  "confidence": "high",
  "one_liner": "Dominant ecosystem with strong cash generation.",
  "red_flags": [],
  "follow_up_questions": ["What is services revenue growth?"],
  "sources": ["https://example.com"]
}
```

"""
result = orchestrator._parse_synthesis_output(test_output)
```

```
assert "Apple shows strong" in result["spoken_briefing"]
assert result["deal_memo"]["verdict"] == "BUY"
assert result["deal_memo"]["company"] == "Apple"
```

def test_audio_pcm_conversion():
"""
Verify audio conversion produces correct PCM format for Gemini Live API.
"""
import numpy as np
from backend.services.audio_utils import chunk_audio

```
# Simulate 1 second of 16kHz mono audio
sample_rate = 16000
duration_secs = 1
num_samples = sample_rate * duration_secs
dummy_audio = np.zeros(num_samples * 2, dtype=np.uint8).tobytes()  # 16-bit = 2 bytes/sa

chunks = chunk_audio(dummy_audio, chunk_size_ms=100, sample_rate=sample_rate)

# 1 second at 100ms chunks = 10 chunks
assert len(chunks) == 10
# Each chunk: 16000 * 0.1 * 2 bytes = 3200 bytes
assert len(chunks) == 3200
```

```
***

## PART 11: TOKEN AND COST CONSTRAINTS

### Token Budget Per Analysis Run

| Component | Model | Est. Input Tokens | Est. Output Tokens | Est. Cost |
|---|---|---|---|---|
| Financial Agent | gemini-2.0-flash | ~800 (prompt + search context) | ~500 (JSON output
| Competitive Agent | gemini-2.0-flash | ~800 | ~500 | ~$0.0003 |
| Sentiment Agent | gemini-2.0-flash | ~800 | ~500 | ~$0.0003 |
| Synthesis Agent | gemini-2.0-flash | ~2500 (all 3 outputs) | ~800 (briefing + memo) | ~
| **Total per analysis** | | **~4900** | **~2300** | **~$0.002** |

**At hackathon (50 demo runs):** ~$0.10 total. Well within free tier.

### Gemini Live API Budget
- Free tier: 1M tokens/month
- One full demo session (~5 min): ~10K tokens
- 100 demo sessions: ~1M tokens → still within free tier

### Rate Limits to Know
- Gemini 2.0 Flash: 15 RPM (requests per minute) on free tier
- Gemini Live API: 3 concurrent sessions on free tier
- If demo fails due to rate limit: add `asyncio.sleep(2)` between agent calls and switch

***
```

```
## PART 12: KNOWN GOTCHAS + FIXES

### Gotcha 1: Live API WebSocket Timeout
**Problem:** Gemini Live API sessions time out after ~10 minutes of silence.
**Fix:** In `live_session.py`, add a keepalive ping every 30 seconds:
```python
async def _keepalive(self):
    while self._session:
        await asyncio.sleep(30)
        await self.send_text(".")  # Silent keepalive
```
```

Gotcha 2: ParallelAgent Race Condition on Shared State

Problem: All three agents write to `session.state` simultaneously. If output keys overlap, one will overwrite another.
Fix: Already handled — each agent has a unique `output_key`: `financial_data`, `competitive_data`, `sentiment_data`. Never reuse keys.

Gotcha 3: ADK + Firestore on Cloud Run Permissions

Problem: Cloud Run service account needs Firestore access.
Fix:

```
gcloud projects add-iam-policy-binding YOUR_PROJECT_ID \
  --member="serviceAccount:YOUR_SERVICE_ACCOUNT@YOUR_PROJECT_ID.iam.gserviceaccount.com
  --role="roles/datastore.user"
```

Gotcha 4: Browser Mic Audio Format Mismatch

Problem: Browser `AudioContext` may default to 48kHz, not 16kHz. Gemini Live API needs 16kHz.
Fix: Already in `useWarRoom.ts` — `AudioContext({ sampleRate: 16000 })`. Also force it in `getUserMedia`: `{ audio: { sampleRate: 16000 } }`.

Gotcha 5: CORS on WebSocket from Firebase Hosting → Cloud Run

Problem: WebSocket connections from Firebase Hosting domain to Cloud Run will fail if CORS isn't set.
Fix: Already in `main.py` — `CORSMiddleware allow_origins=["*"]`. For production, replace `"*"` with your Firebase domain.

Gotcha 6: Synthesis Agent Gets Truncated JSON

Problem: If sub-agent outputs are very long, synthesis agent prompt may exceed context window and truncate the JSON output block.
Fix: Cap each sub-agent output. Add to each agent prompt: `"Keep total output under 500 tokens. Prioritize red flags over comprehensive data."`

Gotcha 7: `adk web` Not Finding Your Agent Module

Problem: Running `adk web` from wrong directory.
Fix: Always run from the root `ma-warroom/` directory, not from `backend/`. The `agent.py` must be discoverable as `backend.agents.orchestrator`.

PART 13: GCP SETUP CHECKLIST

```
# Run these BEFORE the hackathon clock starts

# 1. Create GCP project
gcloud projects create ma-warroom-2026 --name="MA War Room"
gcloud config set project ma-warroom-2026

# 2. Enable required APIs
gcloud services enable \
  run.googleapis.com \
  firestore.googleapis.com \
  aiplatform.googleapis.com \
  cloudbuild.googleapis.com \
  containerregistry.googleapis.com

# 3. Initialize Firestore
gcloud firestore databases create --location=us-central1

# 4. Get Google AI Studio key
# → https://aistudio.google.com/app/apikey
# → Copy key to .env as GOOGLE_API_KEY

# 5. Initialize Firebase
firebase init
# → Select: Hosting, Firestore
# → Use existing project: ma-warroom-2026
# → Public dir: frontend/out
# → Single page app: yes

# 6. Install Python dependencies
cd backend && python -m venv .venv
source .venv/bin/activate  # or .venv\Scripts\Activate.ps1 on Windows
pip install -r requirements.txt

# 7. Install Node dependencies
cd ../frontend && npm install

# 8. Verify ADK installation
adk --version  # Should print version number
```

This is your complete build document. Every file, every line of logic, every gotcha, and every test case you need to vibe-code this into existence in under 6 hours. The architecture is tight, the tech stack is exactly what Google wants to see, and the demo is designed to be jaw-dropping in under 3 minutes. Build it. [][][][][][]